

Тестування

Python-фреймворки для автономного тестування

- **unittest** (стандартна бібліотека, у стилі Java, C#);
- **nose** (для "швидкого" тестування);
- **pytest** (потужний, у пайтонівському стилі).

Основи модульного тестування з unittest

Нехай у файлі `funcs.py` містяться функції, які підлягають тестуванню, тобто, перевірці коректності роботи на різноманітних вхідних даних. Наприклад,

```
# файл funcs.py

def is_letter_in_position(text, position):
    '''Функція перевіряє, чи на позиції position у тексті
    text перебуває літера'''
    return text[position].isalpha()

def mean(*args):
    '''Функція обчислює середнє значення аргументів'''
    return sum(args)/len(args)
```

Продемонструємо основи тестування цих функцій на такому прикладі:

```
# файл test_funcs.py

import unittest
from funcs import *

class FuncsTest(unittest.TestCase):

    def test_is_letter_in_position(self):
        self.assertEqual(is_letter_in_position('xly', 2), True)
        self.assertEqual(is_letter_in_position('xly', 1), False)

    def test_mean(self):
```

```
self.assertEqual(mean(1, 2, 9), 4)

if __name__ == '__main__':
    unittest.main()
```

У цьому файлі створено клас `FuncsTest` на основі базового класу для тестування `TestCase` бібліотеки `unittest`. Власне тести мають вигляд звичайних методів, які мають починатися символами `test` (саме за таким початком метод `main()` шукає тести, які мають бути виконані). У нашому прикладі ми використовуємо єдиний метод перевірки `assertEqual`, якому передаються два аргументи: виклик функції і правильний результат. Якщо функція повертає результат, зазначений у другому аргументі, то тест вважається пройденим.

Результатом виконання нашого коду буде наступне повідомлення:

```
..
-----
Ran 2 tests in 0.052s

OK
```

Методи перевірок

Крім методу `assertEqual`, модуль `unittest` містить багато інших, зокрема:

```
assertNotEqual(a, b)      # a != b
assertTrue(x)             # bool(x) is True
assertFalse(x)            # bool(x) is False
assertIs(a, b)            # a is b
assertIsNone(x)           # x is None
assertIn(a, b)            # a in b
assertNotIn(a, b)         # a not in b
assertGreater(a, b)       # a > b
assertListEqual(a, b)     # перевірка списків на рівність
assertRaises(exc, fun, *args, **kwargs) # fun викликає виняток exc
...
```

Використаємо деякі з них у наших тестах:

```
class FuncsTest(unittest.TestCase):

    def test_is_letter_in_position(self):
        self.assertTrue(is_letter_in_position('xly', 2))
        self.assertFalse(is_letter_in_position('xly', 1))

    def test_mean(self):
        self.assertEqual(mean(1, 2, 9), 4)
        self.assertRaises(ZeroDivisionError, mean)
        self.assertRaises(TypeError, mean, 1, '5')
```

Взагалі кажучи, правильно розроблені тести мають покривати усеможливі випадки вхідних даних.

Інтерфейс командного рядка (CLI)

Часто тести запускаються не з середовища (IDLE), а з командного рядка, особливо досвідченими програмістами. Це пов'язане з тим, що командний рядок дає можливість гнучко маніпулювати параметрами тестування. Розглянемо деякі команди та їх результати:

1) запуск конкретного файлу з тестами

```
$ python3 -m unittest test_funcs.py
```

```
..
```

```
-----
Ran 2 tests in 0.001s
```

```
OK
```

2) запуск конкретного класу з тестами

```
python3 -m unittest test_funcs.FuncsTest
```

```
..
```

```
-----
Ran 2 tests in 0.001s
```

```
OK
```

3) запуск конкретного тесту

```
$ python3 -m unittest test_funcs.FuncsTest.test_mean
```

```
.  
-----  
Ran 1 test in 0.001s
```

OK

4) детальна інформація про проходження тестів

```
$ python3 -m unittest -v test_funcs.py  
test_is_letter_in_position (test_funcs.FuncsTest) ... ok  
test_mean (test_funcs.FuncsTest) ... ok  
-----
```

```
Ran 2 tests in 0.000s
```

OK

5) автоматичний пошук тестів (test discovery)

```
$ python3 -m unittest -v  
test_is_letter_in_position (test_funcs.FuncsTest) ... ok  
test_mean (test_funcs.FuncsTest) ... ok  
-----
```

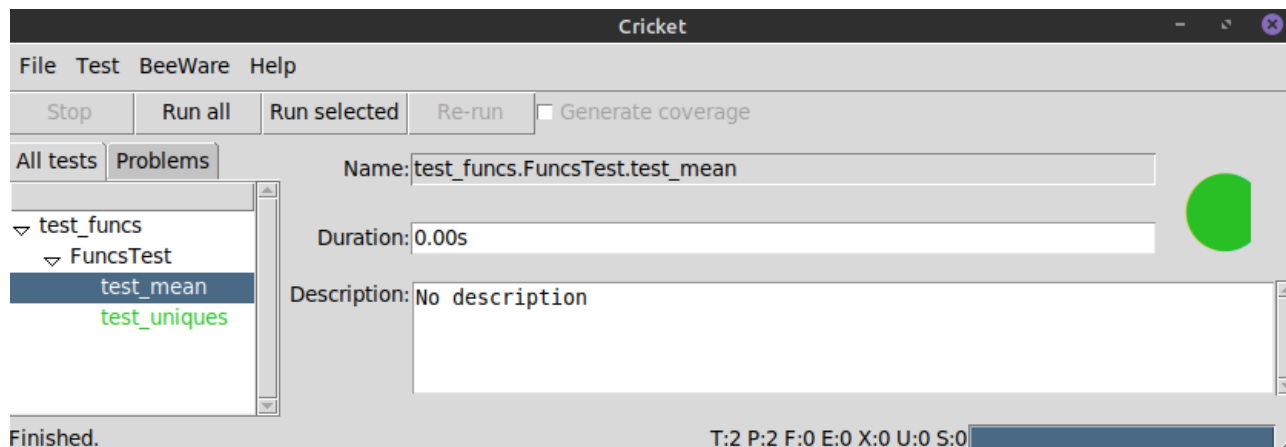
```
Ran 2 tests in 0.001s
```

OK

Графічний інтерфейс для unittest

За бажанням можна додатково встановити модулі, які надають графічний інтерфейс для тестування, наприклад, модуль cricket:

```
$ pip3 install cricket  
$ cricket-unittest
```



setUpClass / tearDownClass

Методи класу setUpClass і tearDownClass виконуються один раз перед та після запуску тестів відповідно.

Зазвичай вони підготовляють складні вхідні дані для тестування та відображають узагальнюючу інформацію. Наприклад, ми могли б підготувати рядки для тестування функції is_letter_in_position(), а також формально вивести інформацію про початок та закінчення тестування:

```
class FuncsTest(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print('---START---')
        cls.s = 'This is a string for test - 1'

    @classmethod
    def tearDownClass(cls):
        print('---FINISH---')

    def test_is_letter_in_position(self):
        '''TEST 1'''
        self.assertTrue(is_letter_in_position('xly', 2))
        self.assertFalse(is_letter_in_position('xly', 1))
        self.assertFalse(is_letter_in_position(FuncsTest.s, -1))

    def test_mean(self):
        '''TEST 2'''
        self.assertEqual(mean(1, 2, 9), 4)
        self.assertRaises(ZeroDivisionError, mean)
        self.assertRaises(TypeError, mean, 1, '5')
```

Як наслідок, матимемо такий результат:

```
$ python3 -m unittest -v
---START---
test_is_letter_in_position (test_funcs.FuncsTest)
TEST 1 ... ok
test_mean (test_funcs.FuncsTest)
TEST 2 ... ok
---FINISH---
```

Ran 2 tests in 0.000s

OK

Як бачимо з результату, для унаочнення можна використовувати також рядки документації.

setUp / tearDown

Методи setUp і tearDown виконуються перед та після запуску кожного тесту відповідно:

```
class FuncsTest(unittest.TestCase):

    counter = 1

    @classmethod
    def setUpClass(cls):
        print('---START---')
        cls.s = 'This is a string for test - 1'

    @classmethod
    def tearDownClass(cls):
        print('---FINISH---')

    def setUp(self):
        print(f'\nStart TEST-{FuncsTest.counter}')

    def tearDown(self):
        print(f'Finish TEST-{FuncsTest.counter}')
        FuncsTest.counter += 1

    def test_is_letter_in_position(self):
        self.assertTrue(is_letter_in_position('xly', 2))
        self.assertFalse(is_letter_in_position('xly', 1))
        self.assertFalse(is_letter_in_position(FuncsTest.s, -1))

    def test_mean(self):
        self.assertEqual(mean(1, 2, 9), 4)
        self.assertRaises(ZeroDivisionError, mean)
        self.assertRaises(TypeError, mean, 1, '5')
```

```
$ python3 -m unittest -v
```

```
---START---
```

```
test_is_letter_in_position (test_funcs.FuncsTest) ...
```

```
Start TEST-1
```

```
Finish TEST-1
```

```
ok
```

```
test_mean (test_funcs.FuncsTest) ...
```

```
Start TEST-2
```

```
Finish TEST-2
```

```
ok
```

```
---FINISH---
```

```
-----  
Ran 2 tests in 0.001s
```

```
OK
```

Провал тесту

Якщо якийсь з методів перевірки не спрацює, тобто, функція поверне неправильне значення, то тест провалиться. Наприклад:

```
def test_mean(self):  
    self.assertEqual(mean(1, 2, 9), 5)  
    self.assertRaises(ZeroDivisionError, mean)  
    self.assertRaises(TypeError, mean, 1, '5')
```

```
$ python3 -m unittest -v
```

```
---START---
```

```
test_is_letter_in_position (test_funcs.FuncsTest) ...
```

```
Start TEST-1
```

```
Finish TEST-1
```

```
ok
```

```
test_mean (test_funcs.FuncsTest) ...
```

```
Start TEST-2
```

```
Finish TEST-2
```

```
FAIL
```

```
---FINISH---
```

```
=====  
FAIL: test_mean (test_funcs.FuncsTest)
```

```
-----  
Traceback (most recent call last):
```

File

```
"/home/mokasin/Документи/COURSES/PROGRAMMING_PYTHON/listings/_11_u
nittest/test/test_funcs.py", line 30, in test_mean
    self.assertEqual(mean(1, 2, 9), 5)
AssertionError: 4.0 != 5
```

```
-----
Ran 2 tests in 0.002s
```

```
FAILED (failures=1)
```

Організація тестування

Зазвичай тести групують у окремі класи, виходячи з тематики функцій, які тестуються, або із структури проєкту. Для прикладу, додамо нову функцію і тест для неї в окремому класі:

```
# файл funcs.py
```

```
...
```

```
def is_positive(x):
    return x > 0
```

```
# файл test_funcs.py
```

```
import unittest
from funcs import *
```

```
class FuncsTest(unittest.TestCase):
    ...
```

```
class OtherTest(unittest.TestCase):
```

```
    def test_is_positive(self):
        self.assertTrue(is_positive(10))
        self.assertFalse(is_positive(0))
```

```
if __name__ == '__main__':
    unittest.main()
```

```
$ python3 -m unittest -v
```

```
---START---
```

```
test_is_letter_in_position (test_funcs.FuncsTest) ...
Start TEST-1
```



```
Finish TEST-1
ok
test_mean (test_funcs.FuncsTest) ...
Start TEST-2
Finish TEST-2
ok
---FINISH---
test_is_positive (test_funcs.OtherTest) ... ok
```

```
-----
Ran 3 tests in 0.001s
```

```
OK
```

TestSuite

Модуль unittest дає змогу групувати класи тестів, які можуть міститися навіть у різних файлах. Група класів, тести з яких мають бути запущені, формують так званий testsuite:

```
# Файл test_runner.py

import unittest
import test_funcs

test = unittest.TestSuite()
test.addTest(unittest.makeSuite(test_funcs.FuncsTest))
test.addTest(unittest.makeSuite(test_funcs.OtherTest))
runner = unittest.TextTestRunner(verbosity=2)
runner.run(test)
```

```
$ python3 test_runner.py
```

```
---START---
test_is_letter_in_position (test_funcs.FuncsTest) ...
Start TEST-1
Finish TEST-1
ok
test_mean (test_funcs.FuncsTest) ...
Start TEST-2
Finish TEST-2
ok
---FINISH---
test_is_positive (test_funcs.OtherTest) ... ok
```

Ran 3 tests in 0.001s

OK

Тестування методів класу

У наведених вище прикладах тестувалися тільки функції. Аналогічно можна тестувати і методи класу. Наведений нижче код демонструє це. Але матеріал цього підрозділу не є обов'язковим для вивчення, а є лише демонстраційним матеріалом для самостійного опрацювання студентами, які хочуть поглибити свої знання в царині модульного тестування.

```
# Файл data.txt
```

```
Roman, 45
```

```
Ann, 38
```

```
Ira, 23
```

```
# Файл person.py
```

```
class Person:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
    def __str__(self):  
        return f"{self.name} ({self.age})"
```

```
# Файл collection.py
```

```
from person import Person
```

```
class Collection:
```

```
    def __init__(self, filename='data.txt'):  
        self.persons = []  
        self.filename = filename  
        self.create()
```

```
    def create(self):  
        with open(self.filename, encoding='utf-8') as f:
```

```

        for line in f:
            parts = line.split(', ')
            name = parts[0].strip()
            age = parts[1].strip()
            self.persons.append(Person(name, age))

    def sort(self):
        self.persons.sort(key=lambda x: x.name)

    def __str__(self):
        return '\n'.join(map(str, self.persons))

```

Файл test.py

```

import unittest
from copy import deepcopy

```

```

class MyTest(unittest.TestCase):

```

```

    @classmethod
    def setUpClass(cls):
        from collection import Collection
        cls.persons = Collection()

    def setUp(self):
        self.persons_for_test = deepcopy(MyTest.persons)

    def test_sort(self):
        self.sorted_names = ['Ann', 'Ira', 'Roman']
        self.persons_for_test.sort()
        for person, name in \
            zip(self.persons_for_test.persons, self.sorted_names):
            self.assertEqual(person.name, name)

```

```

if __name__ == "__main__":
    unittest.main()

```

\$ python3 test.py

.

```

-----
Ran 1 test in 0.016s

```

OK